

```

// Optimal Page Replacement Algorithm in Java

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class OptimalReplacement {
    public static void main(String[] args) throws IOException {

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter the number of frames: ");
        int totalFrames = Integer.parseInt(br.readLine());

        System.out.print("Enter the length of the reference string: ");
        int refLength = Integer.parseInt(br.readLine());

        int[] referenceString = new int[refLength]; // To store the reference pages
        int[] memory = new int[totalFrames];      // To represent frames in memory
        int[][] memoryLayout = new int[refLength][totalFrames]; // For display purpose

        // Initialize memory with -1 (empty)
        for (int i = 0; i < totalFrames; i++)
            memory[i] = -1;

        // Take reference string input
        System.out.println("Enter the reference string:");
        for (int i = 0; i < refLength; i++)
            referenceString[i] = Integer.parseInt(br.readLine());

        int pointer = 0;    // Points to the next frame to replace
        int hitCount = 0;   // Number of hits
    }
}

```

```

int faultCount = 0;    // Number of page faults

boolean isMemoryFull = false;

for (int i = 0; i < refLength; i++) {
    int currentPage = referenceString[i];
    boolean isHit = false;

    // Check if current page is already in memory
    for (int j = 0; j < totalFrames; j++) {
        if (memory[j] == currentPage) {
            hitCount++;
            isHit = true;
            break;
        }
    }

    // If page is not found → Page Fault occurs
    if (!isHit) {
        // If memory is full, apply Optimal Replacement Logic
        if (isMemoryFull) {
            int[] nextUseIndex = new int[totalFrames];    // To store next use position
            boolean[] foundLater = new boolean[totalFrames]; // To mark if page is found later

            // For each page currently in memory, find when it will be used next
            for (int j = i + 1; j < refLength; j++) {
                for (int k = 0; k < totalFrames; k++) {
                    if (referenceString[j] == memory[k] && !foundLater[k]) {
                        nextUseIndex[k] = j;
                        foundLater[k] = true;
                        break;
                    }
                }
            }
        }
    }
}

```

```
    }  
}
```

```
// Replace the page that will not be used for the longest time
```

```
int farthest = -1;
```

```
int replaceIndex = -1;
```

```
for (int j = 0; j < totalFrames; j++) {
```

```
    // If a page is not used again, replace it immediately
```

```
    if (!foundLater[j]) {
```

```
        replaceIndex = j;
```

```
        break;
```

```
    }
```

```
    if (nextUseIndex[j] > farthest) {
```

```
        farthest = nextUseIndex[j];
```

```
        replaceIndex = j;
```

```
    }
```

```
}
```

```
// Replace page
```

```
pointer = replaceIndex;
```

```
}
```

```
// Put the new page into memory
```

```
memory[pointer] = currentPage;
```

```
faultCount++;
```

```
// When frames are not yet full, move the pointer sequentially
```

```
if (!isMemoryFull) {
```

```
    pointer++;
```

```
    if (pointer == totalFrames) {
```

```

        pointer = 0;
        isMemoryFull = true;
    }
}

// Store the current memory layout for display
for (int j = 0; j < totalFrames; j++)
    memoryLayout[i][j] = memory[j];
}

System.out.println("\nMemory Layout (Each column = one step):");
for (int i = 0; i < totalFrames; i++) {
    for (int j = 0; j < refLength; j++) {
        System.out.printf("%3d ", memoryLayout[j][i]);
    }
    System.out.println();
}

System.out.println("\nTotal Hits: " + hitCount);
System.out.println("Total Faults: " + faultCount);
System.out.println("Hit Ratio: " + (float) hitCount / refLength);
}
}

```

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class optimalreplacement {
    public static void main(String[] args) throws IOException
    {
        BufferedReader br = new BufferedReader(new

```

```

InputStreamReader(System.in));

int frames, pointer = 0, hit = 0, fault = 0, ref_len;

boolean isFull = false;

int buffer[];

int reference[];

int mem_layout[][];

System.out.println("Please enter the number of Frames: ");

frames = Integer.parseInt(br.readLine());

System.out.println("Please enter the length of the Reference string: ");

ref_len = Integer.parseInt(br.readLine());

reference = new int[ref_len];

mem_layout = new int[ref_len][frames];

buffer = new int[frames];

for(int j = 0; j < frames; j++)

buffer[j] = -1;

System.out.println("Please enter the reference string: ");

for(int i = 0; i < ref_len; i++)

{

reference[i] = Integer.parseInt(br.readLine());

}

System.out.println();

for(int i = 0; i < ref_len; i++)

{

int search = -1;

for(int j = 0; j < frames; j++)

{

if(buffer[j] == reference[i])

{

search = j;

hit++;

break;

```

```

}
}
if(search == -1)
{
    if(isFull)
    {
        int index[] = new int[frames];
        boolean index_flag[] = new boolean[frames];
        for(int j = i + 1; j < ref_len; j++)
        {
            for(int k = 0; k < frames; k++)
            {
                if((reference[j] == buffer[k]) && (index_flag[k] == false))
                {
                    index[k] = j;
                    index_flag[k] = true;
                    break;
                }
            }
        }
        int max = index[0];
        pointer = 0;
        if(max == 0)
            max = 200;
        for(int j = 0; j < frames; j++)
        {
            if(index[j] == 0)
                index[j] = 200;
            if(index[j] > max)
            {
                max = index[j];
            }
        }
    }
}

```

```

pointer = j;
}
}
}
buffer[pointer] = reference[i];
fault++;
if(!isFull)
{
pointer++;
if(pointer == frames)
{
pointer = 0;
isFull = true;
}
}
}
for(int j = 0; j < frames; j++)
mem_layout[i][j] = buffer[j];
}
for(int i = 0; i < frames; i++)
{
for(int j = 0; j < ref_len; j++)
System.out.printf("%3d ",mem_layout[j][i]);
System.out.println();
}
System.out.println("The number of Hits: " + hit);
System.out.println("Hit Ratio: " + (float)((float)hit/ref_len));
System.out.println("The number of Faults: " + fault);
}
}

```